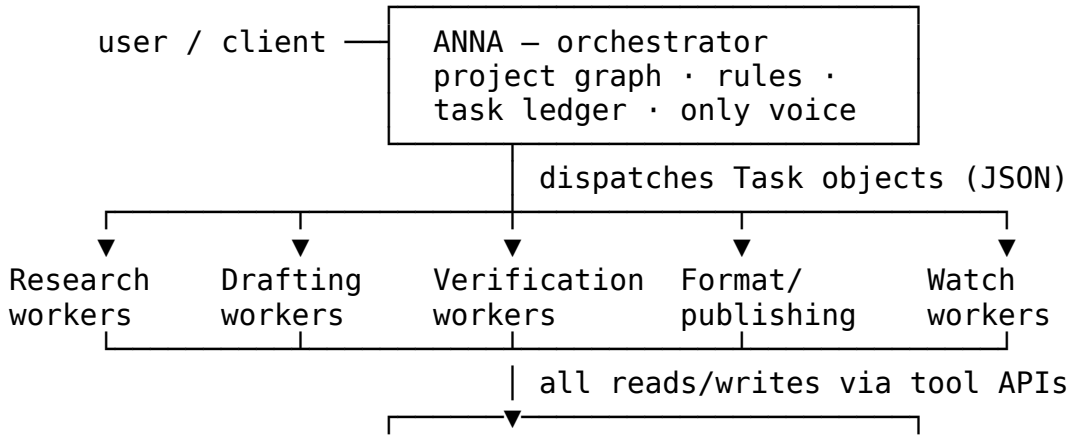
Table of Contents

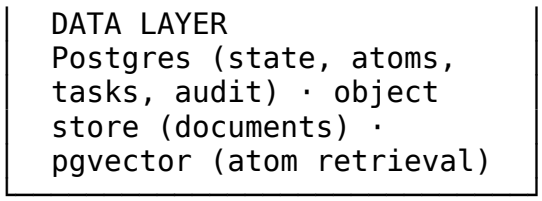
Concept to Cure — Multi-Agent Architecture Spec

Version 0.5 — Nov 6, 2026. Prepared for hand-off to Claude Code as the build specification. Change note: v0.5 — adds §§16–19 (trust architecture: credential custody, client-owned authority accounts, and how the pattern AnA works under becomes a product feature set), §§20–23 (continuous evolution: how Anna learns, sets rules, and hardens guardrails — per user, per customer, per project), §§24–26 (data locality: classification-driven routing — raw data stays home, reasoning travels on derivatives), §§27–29 (three kinds of learning: what never crosses tenants, what crosses de-identified, and Anna's craft, which compounds globally), and §§30–32 (agency-feedback intelligence: learning objective and subjective reasons agencies push back, and the clean-file review that acts on them before transmission). §§1–15 are unchanged from v0.4. Earlier increments: v0.4 added §14 (AnA-in-product feasibility investigation) and §15 (Claude Code handoff / roadmap brief); v0.2 added §12 (platform topology); v0.3 added §13 (Anna runtime + leverage map) and §13.5 (decision rights). **One-line design:** Anna (the orchestrator) holds the client, the project graph, the rules, and the memory. Stateless specialist workers execute narrow tasks in parallel against a provenance-first data layer. Deterministic code computes every number; LLM agents handle language; nothing reaches the client without passing a verification gate.

---

1. Architecture at a glance





Non-negotiable principles 1. **Provenance first.** Every claim in every document is an *atom* carrying its source (document, page, span) or its generator (code hash + dataset hash). No atom, no sentence. 2. **Deterministic vs. LLM split.** Numbers, diffs, threshold checks, string-exact citation matches = code. Drafting, synthesis, interpretation = LLM agents. Never let an LLM compute a number that code can compute. 3. **Drafter ≠ checker.** Verification is always a different agent (and where possible a different mechanism — recompute, not re-read). 4. **Workers are stateless and sandboxed.** A worker receives a Task + input bundle, returns an artifact + self-report, and can write only to its task sandbox. Anna merges. 5. **Judgment escalates.** Workers execute inside a defined envelope. Regulatory strategy calls, exceptions, and client commitments route to the human through Anna.

---

2. Core data model (Postgres; JSONB where noted)

2.1 Project

```
Project {
  id, client_id,
  client_type: "biotech" | "pharma" | "device_dx",
  product: { name, modality, indication },
  jurisdictions: ["FDA-CDER" | "FDA-CBER" | "FDA-CDRH" | "EU-MDR" |
"EMA" ...],
  document_set: [DocumentNodeId],          // the project graph roots
  ctq_factors: [CtQ],                      // critical-to-quality
factors (drives RBM, §7)
  status, created_by, audit_ref
}
```

2.2 DocumentNode — the project graph (a DAG, not a folder tree)

```
DocumentNode {
  id, project_id, parent_id,
  kind: "IND" | "CSR" | "CTD-M1..M5" | "510K" | "CER" | "IB" |
"PROTOCOL" ...,
  section_ref: "3.2.P.5.1" | "CSR-11.4.1" | "eSTAR-device-description"
...,
  state: "planned" | "inputs_ready" | "drafting" | "verifying" |
"locked" | "published",
```

```
  depends_on: [AtomId | DocumentNodeId],   // a node unlocks when deps
verify
  owner_agent_role, version, content_hash
}
```

The graph is the scheduler: a section becomes dispatchable the moment every atom and upstream section it depends on reaches `verified`/`locked`.

2.3 Atom — the unit of truth

```
Atom {
  id, project_id,
  claim_type: "endpoint" | "result" | "number" | "method" |
"statement"
            | "batch_result" | "stability_point" | "lab_flag" |
"kri_value" ...,
  text,                                    // the claim as written
language
  value: { num, unit_ucum, normalized },  // for numeric atoms; UCUM
units only
  provenance: {
    kind: "source" | "computed",
    source_doc_id, page, span_start, span_end,        // kind=source
    generator: { code_hash, dataset_hash, run_id }    // kind=computed
  },
  jurisdiction_tags: [],                   // an atom valid for FDA may
not be for EU
  verification: { status: "unverified" | "verified" | "rejected",
                  method: "string_match" | "recompute" | "agent_check"
| "human",
                  verified_by, verified_at, evidence_ref },
  reuse: [DocumentNodeId]                  // every place this atom
appears
}
```

Born-verified rule: atoms produced by deterministic pipelines (stats runner, stability regression, KRI engine) are born `verified` with `method: "recompute"` available on demand — their provenance is the code + data hash, so any checker can re-derive them exactly.

2.4 ProtocolContract — extracted from the protocol by the Protocol Analyst agent, human-locked

```
ProtocolContract {
  project_id, protocol_version,
  objectives: [{ id, text, atom_refs }],
  endpoints: [{ id, tier: "primary" | "secondary" | "exploratory",
                definition, analysis_population, estimand_ref }],
  hypotheses: [{ id, endpoint_id, null, alternative, alpha,
```

```
multiplicity_rule }],
  sap_ref, populations: ["ITT" | "mITT" | "PP" | "SAFETY" ...],
  locked_by, locked_at                    // GATE: no drafting before
this lock
}
```

2.5 Task — the unit of work Anna dispatches

```
Task {
  id, project_id, node_id,
  role: "atom_curator" | "drafter" | "verifier" | "stats_runner" |
"qc_redteam" ...,
  inputs: { atom_ids: [], doc_ids: [], params: {} },
  state: "queued" | "running" | "self_check_failed" | "verifying" |
"done" | "failed" | "escalated",
  idempotency_key, attempts, sandbox_uri, output_ref, self_report,
  created_by: "anna" | "scheduler" | "watch_event"
}
```

2.6 Finding — QC output

```
Finding { id, node_id, severity: "blocker" | "major" | "minor",
          kind: "contradiction" | "bad_citation" | "number_mismatch" |
"missing_input" ...,
          detail, atom_refs, status: "open" | "resolved" |
"accepted_by_human" }
```

---

3. Worker contracts (tool APIs)

Every worker is a function: `(Task, InputBundle) → (Artifact, SelfReport)`. Workers call only these tools:

Tool	Purpose
`dataroom.list / read / catalog`	Evidence ingest and content-based catalog (never trust filenames)
`atoms.search(query, filters)`	Retrieve verified atoms (pgvector + structured filters)
`atoms.propose(bundle)`	Submit new atoms for verification (workers never self-verify)
`atoms.verify(atom_id, method)`	Verifier-role only: string-match, recompute, or agent-check
`draft.get_node / put_section`	Read/write a section in the worker's sandbox

Tool	Purpose
`compute.run(job_spec)`	Deterministic execution: stats, stability, KRI, diffing (§5–§7)
`render.check(node_id, format)`	Format validation: eCTD granularity, eSTAR mapping, CER structure
`findings.raise(finding)`	Any worker can raise a Finding; blockers halt the node

Agent roles (initial roster): Project Architect · Protocol Analyst · Evidence Librarian · Atom Curator · Literature Agent (CER/search-and-appraise) · Guidance Agent · Section Drafter · Consistency Agent (cross-document contradiction hunt) · Citation Verifier · Format Agent (one per jurisdiction family) · QC Red-Team · Watch Agent (scheduled).

Orchestrator loop (Anna): maintain graph state → find unlocked nodes → compose Tasks with the minimal input bundle → dispatch (parallel, bounded concurrency per role) → route outputs to verification → merge verified work → escalate Findings and judgment calls → report to the user. Anna is the only component with user contact and the only writer to `locked` state.

---

4. Document factories by client type

Each factory is a **graph template** the Project Architect instantiates. Sections unlock in parallel as their input atoms verify.

4.1 Biotech — IND + CSR

- **IND graph:** 1.6/1.7 administrative & presubmission · Module 2 summaries (2.3 quality, 2.4 nonclinical, 2.5 clinical overview — skeletons early) · Module 3 (see §6) · Module 4 reports · 5.2 tabular listing · protocol + amendments · IB.
- **CSR graph (ICH E3):** synopsis → §§9–15. The CSR is the platform's *atom factory*: its results tables become computed atoms reused by the IB, protocol amendments, Module 2.5/2.7, and the eventual marketing application. CSR sign-off (statistician/medical writer signatures) is a QMS-controlled event (§8).

4.2 Pharma — full CTD

- Five modules; **Module 2 is drafted from Modules 3–5 atoms, never free-standing** (graph dependency enforces this).
- Cross-document consistency is a first-class job: the Consistency Agent continuously diffs shared atoms (dose, exposure, AE counts) across every module that reuses them; a changed atom raises Findings on every dependent node.

4.3 Device / diagnostics — 510(k) via eSTAR + EU CER

- **510(k) graph = eSTAR's own structure** (device description, substantial equivalence, performance data…). The SE argument is a typed atom set: predicate characteristics vs. subject device, field-by-field, each cell provenanced. The Format Agent maps section nodes → eSTAR fields 1:1 and emits an attachment manifest (see §9 publishing).
- **CER graph (EU MDR Annex XIV):** state-of-the-art, literature search protocol → screening atoms (include/exclude + reason) → appraisal (MDCG 2020-13 style weighting) → clinical evidence synthesis → benefit-risk. Literature work is the heavy parallel stream here.

---

5. Feature module — Biostatistics

Design rule: the statistics pipeline is *code that agents operate*, not agents doing math.

Component	What it does
SAP Parser agent	Extracts the Statistical Analysis Plan into machine form: analyses, populations, estimands (ICH E9(R1) five attributes), multiplicity, missing-data rules → binds to ProtocolContract.hypotheses. Discrepancies SAP↔protocol raise Findings.
Dataset intake	CDISC SDTM/ADaM validation (structure, controlled terminology, define.xml cross-check) via `compute.run` validators; dataset hash pinned.

Component	What it does
Analysis Runner	Executes versioned, reviewed analysis programs (R/Python, containerized, pinned package lockfile) against the pinned dataset hash. Outputs TLFs (tables/listings/figures). Every output cell that matters becomes a **computed atom** (code hash + dataset hash provenance).
Independent QC (double programming)	A second runner re-derives key TLFs from an independently written program spec; `compute.diff` compares cell-by-cell. Any mismatch = blocker Finding. This is the industry-standard QC pattern, mechanized.
Stats Writer agent	Drafts CSR results narrative (§§11–12) and Module 2.7.3 text *only* from verified TLF atoms; cannot type a number that is not an atom.
Estimand/consistency check	Deterministic cross-check: endpoint definitions identical across protocol, SAP, CSR, and CTD summaries (string + structure diff on the contract objects).

Acceptance test for Claude Code: fixture ADaM dataset + fixture SAP → runner reproduces the fixture TLF exactly; seeded one-cell change in the QC program → diff catches it.

---

6. Feature module — CMC / Module 3 (Chemistry, Manufacturing, and Controls)

Module 3 is data-dense and table-dense, so the design centers on **typed atoms + generated tables**: tables are rendered from atoms by code, never hand-typed — transcription error becomes structurally impossible.

6.1 Structure (instantiated per product)

- **3.2.S drug substance:** S.1 general info · S.2 manufacture · S.3 characterisation · S.4 control (specs, analytical procedures, validation, batch analysis, justification) · S.5 reference standards · S.6 container closure · S.7 stability.
- **3.2.P drug product:** P.1 description/composition · P.2 pharmaceutical development · P.3 manufacture · P.4 excipient control · P.5 control of drug product · P.6 reference standards · P.7 container closure · P.8 stability.
- **3.2.A** appendices · **3.2.R** regional · **3.3** literature.

6.2 Typed CMC atoms

```
BatchAtom      { batch_id, scale, site, date, stage }
// S.2/P.3, batch tables
SpecAtom       { attribute, method_ref, acceptance_criterion, phase }
// S.4.1/P.5.1 tables
ResultAtom     { batch_id, attribute, value, unit, spec_ref, oos: bool
} // S.4.4/P.5.4 batch analyses
StabilityAtom  { batch_id, condition, timepoint_mo, attribute, value }
// S.7/P.8 datasets
MethodAtom     { method_id, technique, validation_ref }
// S.4.3/P.5.3
```

6.3 CMC agents

- **Batch Data agent** — ingests CoAs / batch records / LIMS exports → BatchAtom + ResultAtom sets; OOS/OOT results auto-flagged by `compute.run` against SpecAtoms.
- **Stability agent** — builds the stability dataset; `compute.run` performs ICH Q1E analysis (regression, poolability, shelf-life estimation) → computed atoms + generated charts; drafts S.7.3/P.8.3 narrative from them.
- **Specs & Methods agent** — maintains the spec tables as atom sets; checks every method cited exists as a MethodAtom with validation; justification sections (S.4.5/P.5.6) drafted against the actual data distribution (computed summary stats).
- **Comparability agent** — on any process/site/scale change: diffs pre/post ResultAtom distributions, highlights shifted attributes, drafts the comparability protocol/report skeleton. (This is also the engine behind post-approval change supplements, §9.4.)
- **Module 3 Format agent** — eCTD granularity enforcement (leaf per document at the right node), CTD numbering, cross-references to Module

2.3 (QOS) generated from the same atoms so QOS can never drift from Module 3.

Acceptance test: fixture batch + stability spreadsheets → generated S.4.4 batch-analysis table and P.8 stability table match expected output cell-for-cell; a seeded OOS value is flagged, never silently tabled.

---

7. Feature modules — Lab data, QNS, and Risk-Based Monitoring

7.1 Central lab pipeline + QNS

- **Intake:** lab feeds (flat file / HL7 / FHIR) → normalization by code: LOINC test codes, UCUM unit conversion, reference-range harmonization per lab/age/sex. Raw feed preserved immutably; normalization is a versioned transform (provenance).
- **Flag engine (`compute.run`, deterministic):** critical values, out-of-range, hemolysis/icterus flags, and **QNS — quantity not sufficient** (sample insufficient for one or more tests).
- **QNS as a first-class object:**

```
LabFlag { id, subject_id, visit, test_code, flag: "QNS" | "CRITICAL" |
"HEMOLYSED" ...,
          source_feed_hash, status: "open" | "recollected" |
"waived_by_medical" | "closed",
          sla_due, site_id }
```

A QNS opens a tracked workflow: recollection task to the site, SLA clock, escalation. QNS rate per site/test is computed continuously and **feeds RBM (§7.2)** — a site with rising QNS is a quality signal, not an anecdote. - **Safety signal agent:** shift tables and liver-signal screening (e.g., Hy's law quadrant plots) generated by code from normalized data; narrative drafted by agent from computed atoms only.

7.2 Risk-Based Monitoring / RBQM (ICH E8(R1) + E6(R3))

- **Setup:** CtQ factors declared on the Project (§2.1). Each CtQ maps to one or more **Key Risk Indicators**.
- **KRI engine (`compute.run`, scheduled):** deterministic metrics from live operational data — enrollment velocity, screen-failure rate, protocol-deviation rate, query rate and aging, **QNS rate**, AE reporting lag, data-entry lag, missing-visit rate. Thresholds are config (green/amber/red per KRI per study phase).

```
KRIValue { kri_id, scope: "study" | "site" | "country", site_id?,
period,
           value, threshold_state, computed_from: [dataset_hash...],
at }
```

- **Signal agent (LLM, on threshold breach):** interprets the breach pattern across KRIs, drafts the central-monitoring note, proposes targeted SDV/SDR scope changes and site follow-up tasks. It proposes; the monitoring plan change itself is a QMS change-control event (§8).
- **Central Monitoring Report drafter:** assembles the periodic report from KRIValue atoms — every number in the report is a computed atom, so the report is reproducible for any period on demand.

Acceptance test: fixture operational dataset with a seeded deteriorating site → KRI engine turns that site's QNS + query-aging KRIs amber/red on the right periods; central monitoring report regenerates with identical numbers.

---

8. Canvas editor, audit, and Part 11 layer

- **Document = live tree of DocumentNodes.** Each section renders its atoms as inline, inspectable references (click a number → its provenance card: source page or code+data hash).
- **Agent proposals arrive as diffs.** Accept/reject per hunk; rejected hunks return to the drafter with the human's note attached to the Task.
- **Locks are hash-chained versions.** Section lock = content hash + locker identity + timestamp appended to an immutable audit log (who, what, when, before/after hashes) — the 21 CFR Part 11 / EU Annex 11 pattern: e-signatures with meaning-of-signature ("authored", "reviewed", "approved"), two-component signing for approval events, and a complete, exportable audit trail per project.
- **Roles & permissions:** author / reviewer / approver / QA segregated; workers hold task-scoped write tokens only; Anna holds no signing key — signatures are human-only events.

---

9. Feature module — QMS (Quality Management System)

The QMS is two things at once: (a) the quality system **the platform itself runs under** (so its outputs are inspection-defensible), and (b) a **client-facing module** —

device clients live under ISO 13485 / FDA QMSR (21 CFR 820) and pharma/biotech under GxP; the same objects serve both.

9.1 QMS objects

```
ControlledDocument { id, type: "SOP" | "template" | "work_instruction"
| "form",
                     version, effective_date, supersedes, owner,
state:
                     "draft" | "in_review" | "approved" | "effective"
| "obsolete" }
ChangeControl      { id, scope, rationale, impact_assessment: { nodes:
[], atoms: [] },
                     approvals: [], state, linked_submission_seq? }
Deviation          { id, classification: "minor" | "major" |
"critical", description,
                     root_cause, linked_capa? }
CAPA               { id, source_deviation?, actions: [{ owner, due,
evidence }],
                     effectiveness_check: { due, criteria, result } }
TrainingRecord     { user, document_ref+version, completed_at,
assessment? }
Audit              { id, kind: "internal" | "supplier" |
"mock_inspection", findings: [] }
```

9.2 How QMS wires into everything else

- **Templates are controlled documents.** A drafting agent may only instantiate a section from an *effective* template version; the template version is recorded on the node (so any produced document can answer "which template, which version" in an inspection).
- **Every lock/approval in §8 is a QMS event** with training prerequisites checked: a user cannot approve a document type their TrainingRecord doesn't cover.
- **Deviations inside production work** (e.g., a section shipped to internal review with an unverified atom) are raised as Deviation objects by the QC Red-Team automatically — the platform audits itself.
- **Change control drives submissions:** a change to a specification, process, or label opens a ChangeControl; its impact assessment enumerates affected atoms/nodes deterministically (atom reuse graph), and Anna proposes the regulatory reporting category (e.g., PAS / CBE-30 / annual report for drugs; new 510(k) vs. letter-to-file analysis scaffold for devices) **as a human decision** — category selection is a judgment call and is never automated.

- **Supplier quality:** CROs, labs, and CMOs are qualified suppliers with audit schedules — which is where the RBM site signals (§7.2) and lab QNS rates (§7.1) land as supplier-performance evidence.

9.3 Inspection readiness

One-click **inspection pack** per project: audit trail export, training matrix, controlled-document register, deviation/CAPA log with effectiveness checks, and the atom-provenance index for any sampled claim. If it can't be exported, it didn't happen — build the exporters with the features, not after them.

---

10. Feature module — Publishing, FDA submission, and the correspondence loop

This is the full downstream lifecycle: **publish → validate → transmit → acknowledgment → agency correspondence → response sequences**. It is a state machine, not a file export.

10.1 Publishing (per format)

- **eCTD publisher:** renders section nodes to leaf PDFs (PDF spec per FDA: searchable text, bookmarks/hyperlinks per granularity rules, no security, fonts embedded), generates the XML backbone (index.xml + regional M1), assigns leaf metadata (titles, operations: new/replace/delete/append), computes MD5 checksums, and emits the sequence folder (`0000`, `0001`, …) in us-eCTD structure.
- **Validation gate (`compute.run`):** an internal validator implementing the FDA eCTD validation criteria (high/medium/low error classes per the current Technical Conformance Guide): broken links, checksum mismatch, wrong granularity, PDF non-conformance = **blockers — a sequence with a high-severity error cannot be transmitted**. Validation report is a QMS record.
- **eSTAR (510(k)):** the publisher maps node content into eSTAR's structured fields and attachment slots (§4.3), validates completeness against eSTAR's own checks, and produces the eSTAR package for upload; eSTAR version changes are handled as a Format Agent template update under change control.
- **EU/device technical documentation:** structured assembly per MDR Annexes II/III with the CER (§4.3) as a component; notified-body submission packages generated from the same atoms.

10.2 Submission object and lifecycle

```
Submission {
  id, project_id, application: { type: "IND" | "NDA" | "BLA" | "510k"
| "PMA" | "MDR-TD",
                                  number?, sponsor },
  sequences: [ { seq_no, kind: "original" | "amendment" | "supplement"
| "response",
                 content_nodes: [], validation_report_ref, state } ],
  gateway: { channel: "ESG" | "CDER-Direct" | "eSTAR-portal" | "NB-
portal",
             transmitted_at, ack1?, ack2?, ack3?,      // gateway +
center receipts
             core_id?, message_id? }
  state: "drafting" | "publishing" | "validating" | "ready" |
"transmitted"
       | "acknowledged" | "under_review" | "action_received" |
"closed"
}
```

- **Transmission happens only from `ready`** (validation passed + human approval signature in QMS). Anna prepares; the human transmits or explicitly authorizes transmission. Gateway credentials live in a vault, never with workers.
- **Acknowledgment tracking:** ESG acknowledgments (receipt → center acknowledgment → review-center assignment) are ingested as events on the Submission; a missing ACK2/ACK3 inside its expected window raises a Watch task automatically.

10.3 Correspondence intake — the other half of the loop

FDA writes back through many doors (ESG/CDER Direct notices, email, mail, portal). All of it lands in **one intake**:

```
Correspondence {
  id, submission_id, received_at, channel, raw_object_uri,
  classified: { kind: "ACK" | "RTF" | "RTA" | "AI_REQUEST" |
"DEFICIENCY"
                    | "CRL" | "APPROVAL" | "MEETING_MINUTES" |
"INFO_REQUEST" | "OTHER",
                confidence, classifier_agent, human_confirmed: bool },
  clocks: [ { kind: "response_due", due_at, basis: "regulation" |
"letter_text",
              extracted_from_span } ],          // every deadline is
an object with a source
  response_plan: { nodes: [], tasks: [] },      // Anna builds this on
classification
  state: "received" | "triaged" | "responding" | "responded" |
```

```
"closed"
}
```

- **Classification agent** reads the full letter (never the subject line), extracts action items and every date, and proposes the kind + clocks. **Dates are atoms** — extracted with the letter span as provenance, human-confirmed before any clock starts.
- **The response workflow is pre-wired per kind:**
  - **RTF (drugs) / RTA (devices):** response plan targets the cited deficiencies item-by-item; each item maps to nodes/atoms to fix or argue; resubmission = new sequence referencing the original.
  - **AI request / interactive review (510(k), IND/NDA amendments):** deficiency list → one response node per deficiency → owners assigned → response package assembled and published as a new sequence inside the clock; partial-response rules flagged for human decision.
  - **CRL:** escalates to a program-level plan (Type A meeting request drafting, resubmission strategy as human decisions; agents prepare options, never choose).
  - **Meeting minutes:** action items extracted → tasks on the project graph; commitments made by the agency are pinned as atoms so later documents can cite them.
- **Clock discipline:** every open clock appears on Anna's daily ledger with days remaining; amber at 50% elapsed, red at 75%; a response sequence cannot miss silently — the same "never let it linger" rule the user enforces in email, enforced by the scheduler.

10.4 Post-approval lifecycle

Approved applications stay alive in the graph: supplements (PAS/CBE/annual-reportable via §9.2 change control), annual reports (assembled from the year's change/deviation atoms), periodic safety reporting inputs, and device post-market (MDR reports flow through the same Correspondence intake with kind `INFO_REQUEST/DEFICIENCY` mapping). The submission history is one queryable chain: application → sequences → correspondence → responses → approvals.

Acceptance tests: fixture eCTD sequence → validator passes a clean build and blocks a build with a seeded broken hyperlink; fixture AI-request letter → classifier extracts the correct kind, all action items, and the response date with span provenance; ACK simulator → missing ACK3 raises a Watch task.

---

11. Stack and build order (for Claude Code)

Suggested stack: TypeScript monorepo (or Python if the stats sandbox leads) · Postgres + pgvector · S3-compatible object store · containerized compute workers (R/Python for stats; deterministic services for validation/KRI) · a durable workflow engine (Temporal or DB-backed queue) for the Task DAG · LLM API with tool use for agents · append-only audit log (hash-chained).

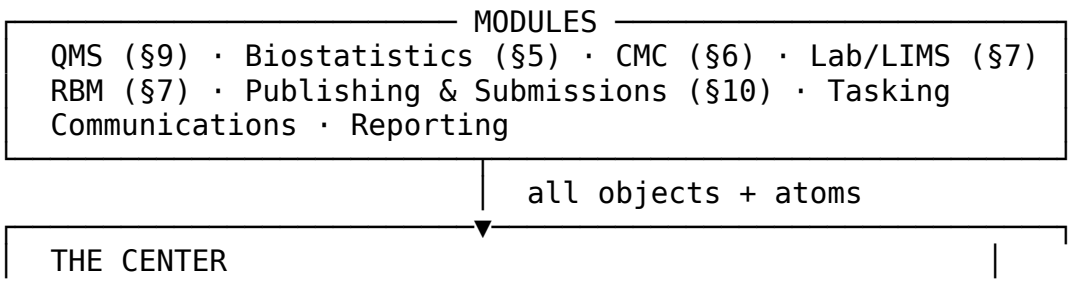
Build order — each phase ships with its acceptance test green before the next starts: 0. **Skeleton:** schemas (§2), Task engine, sandbox runner, audit log, one dummy worker end-to-end. 1. **Data room + atoms + verifier.** Test: seeded bad citation is caught; atoms carry provenance end-to-end. 2. **Canvas + CSR drafting** (one document type, full gate flow). Test: a section cannot lock with an unverified atom. 3. **Biostatistics** (§5). Test: fixture TLF reproduced exactly; seeded QC diff caught. 4. **CMC Module 3** (§6). Test: fixture batch/stability data → exact tables; OOS flagged. 5. **Lab/QNS + RBM** (§7). Tests per §7. 6. **QMS** (§9) wired to templates, locks, and change control. Test: unapproved template version cannot be instantiated; inspection pack exports. 7. **Publishing + submission + correspondence** (§10). Tests per §10.4. 8. **510(k)/eSTAR + CER factories** (§4.3) on top of the now-proven spine.

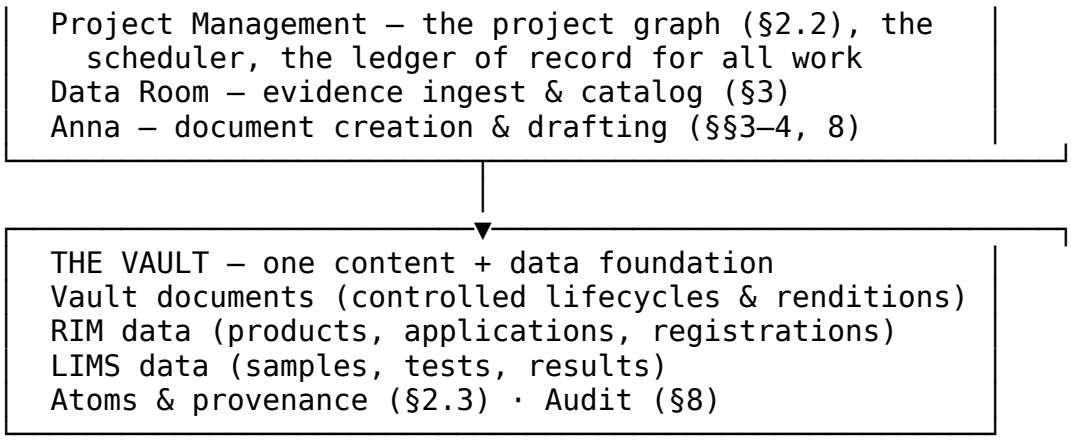
Hand-off format for the build: this spec + one fixture project per client type (anonymized/synthetic: IND with CSR, CTD skeleton, 510(k) with predicate) as golden test data; per-agent prompt files versioned as controlled documents inside the repo (the platform's QMS discipline applies to its own prompts).

---

12. Platform topology — the Vault, the center, and the modules (v0.2 addition)

Everything in §§1–11 sits on one topology. At the center is a triad; around it, a Vault foundation; around that, the feature modules. Nothing is a silo: every module reads and writes the same Vault objects and the same atom layer.





12.1 The Vault (Veeva-class content + data foundation)

- **VaultDocument** — the single document object for the whole platform:

```
VaultDocument { id, vault_no, title, type, subtype, classification,
  lifecycle: "Draft" | "In Review" | "Approved" | "Effective" |
"Superseded" | "Obsolete",
  version: { major, minor }, steady_state_version,
  source_file_uri, renditions: [{ kind: "viewable" | "submission_pdf"
| "native", uri, hash }],
  owner, steward, security: { roles: [], acl: [] },
  based_on_template: ControlledDocumentRef,        // QMS link (§9.2)
  project_refs: [], submission_refs: [] }
```

Lifecycle transitions are workflow events (review/approve tasks auto-created by the Tasking module) and QMS events (Part 11 e-signatures, §8). Every DocumentNode in a project graph (§2.2) *is* a VaultDocument in a drafting lifecycle state — one object, two views (project view, vault view). No duplicate stores. - **Renditions are generated, never uploaded twice:** the canvas edits the source; the viewable and submission renditions are rendered by code (this is what §10.1 publishing consumes). - **Full-text + metadata search across the vault**, with the content-based cataloging rule (§3): classification comes from the document's contents, not its filename.

12.2 RIM data (registrations & product intelligence — the Veeva RIM analog)

```
Product        { id, names: [], modality, substances: [],
manufacturer_sites: [] }
Application    { id, product_ref, authority, type, number, status }
// IND/NDA/510(k)/MDR family
Registration   { id, application_ref, market, status,
                 commitments: [{ text, due, status }],        // post-
approval commitments as tracked objects
                 renewals: [{ kind, due }] }                   //
```

```
annual reports, PADER/PSUR-type clocks
SubmissionPlan { application_ref, planned_sequences: [{ kind,
target_date, readiness }] }
```

- RIM is the system of record §10's Submission objects register into: a Submission (§10.2) is an *event*; the Application/Registration is the *standing record* it updates. Correspondence (§10.3) attaches to both.
- **Registration intelligence:** commitments and renewals carry clocks on Anna's ledger with the same amber/red discipline as response clocks (§10.3). Product data is aligned to IDMP concepts (substance/product/organisation/referential) so EU submissions don't require a second data model.

12.3 LIMS data (the Veeva/LIMS analog — feeds §7)

```
Sample { id, subject_ref?, study_ref, type, collected_at, received_at,
condition_on_receipt,
         state: "received" | "in_test" | "resulted" | "flagged" |
"closed" }
Test   { id, sample_ref, loinc, method_ref, state }
Result { id, test_ref, value, unit_ucum, range_ref, flags: ["QNS" |
"CRITICAAL" ...],
         provenance: { feed_hash, transform_version } }
```

The §7.1 flag engine operates on these objects; a QNS flag is a state on a Test with the recollection workflow attached (§7.1 LabFlag is the workflow view of this state). Stability samples (§6.3) are LIMS samples too — one lab data spine for clinical and CMC.

12.4 Tasking module

The §2.5 Task generalizes into the platform's single work engine — **human and agent tasks in one system**:

```
WorkTask { id, kind: "agent" | "human" | "approval" | "review",
           assignee: AgentRole | UserId | QueueId, project_ref,
node_ref?,
           sla: { due, warn_at }, checklist: [], state, history: [] }
```

- Every workflow in the platform is WorkTasks: QMS approvals, canvas reviews, correspondence response items (§10.3), RBM follow-ups (§7.2), QNS recollections (§7.1), submission readiness checks (§10.2).
- Queues by role with SLA clocks; escalation on breach to Anna's ledger. A user sees one inbox: their human tasks across all projects and modules.

12.5 Communications module

One communications log for the platform — not an email clone, a **record**:

```
CommRecord { id, project_ref?, submission_ref?, thread_id,
             direction: "in" | "out", channel: "gateway" | "portal" |
"email" | "letter" | "meeting",
             parties: [], at, raw_ref, summary_atom_refs: [],
             acknowledgment: { required: bool, sent_at? } }
```

- FDA correspondence (§10.3) is the regulated subset of this log; client and internal communications live in the same structure with different security classes.
- **Acknowledgment discipline is built in:** inbound records that require acknowledgment open a WorkTask until the acknowledgment CommRecord exists (the platform never lets a message sit unanswered — §10.3's rule, generalized).
- Meeting records attach here too; agency meeting minutes become CommRecords whose commitments are atoms (§10.3).

12.6 Reporting module

- A metrics layer over Vault + atoms + WorkTasks + Submissions. **Every report is reproducible:** a report is a stored query + the hashes of its inputs, so any historical report can be regenerated exactly (inspection-grade).
- Prebuilt families: project status & burn-down (nodes by state, blockers by age) · submission pipeline (per application, per market, clocks) · quality (deviations, CAPA aging, QNS rates, KRI heatmap) · workload (queues, SLA breaches) · registration/commitment calendar.
- Ad hoc: Anna composes reports from the same layer on request — narrative reports are drafted by an agent from computed atoms, exactly like §7.2's central monitoring report.

12.7 What this topology buys (state it in the build kickoff)

1. **One object model.** A stability result, a QNS flag, a deficiency letter, and a CSR table are all Vault/atom objects — cross-module features (RBM reading LIMS, change control reading submissions) are queries, not integrations.
2. **The center stays small.** Project Management + Data Room + Anna drafting is the product's heart; modules plug into it without changing it.
3. **Regulated by construction.** QMS lifecycle, Part 11 audit, provenance, and acknowledgment/clock discipline are platform services every module inherits — not features each module rebuilds.

13. Anna, implemented — and the third-party leverage map (v0.3 addition)

13.1 Designing the orchestrator (Anna) as a runtime, not a prompt

Anna is not one big prompt. She is a **runtime with six components**, each mapping to a platform service:

Anna component	What it is in the product	Implementation notes
Persistent matter memory	Everything known about a client/project survives sessions: facts, decisions, corrections, standing rules	Curated memory store (structured records + summaries) per client/project, versioned; loaded into context at session start; the runtime *writes back* durable learnings. This is the component commodity chatbots lack — build it first-class.
Skill library	Repeatable playbooks: "build a CSR," "answer an AI request," "run the stability analysis"	Skills are files (procedure + tool grants + gates), versioned **as controlled documents in the QMS (§9)** — a skill change is a change-control event. Anna loads the skill matching the task type, nothing more.
Subagent spawner	Dispatches stateless workers (§3) with self-contained briefs: task, outcome, constraints, input bundle	Workers inherit the brief, not the conversation. Results return to Anna; Anna verifies and merges. Concurrency bounded per role; every spawn is a WorkTask (§12.4).

Anna component	What it is in the product	Implementation notes
Tool & connector layer	Vault, data room, compute, publishing, gateway, email/calendar, browser	Connectors are scoped credentials from the **secure vault** (§13.2); workers get task-scoped tokens, never raw credentials.
Scheduler / watchers	Cron-like jobs + event hooks: KRI runs, clock checks, ACK watches, guidance monitoring	Every watcher is a WorkTask factory with an owner and a stop condition (§10.3 clock discipline runs on this).
Approval gates	Runtime-enforced pauses: signature, transmission, payment, judgment calls	Gates are *platform* objects, not prompt text — a worker cannot talk its way past one. The human's approval is a signed QMS event (§8).

Design rule that follows: the moat is the memory + skills + gates, not the model. Models are swappable underneath Anna; the matter memory, the controlled skill library, and the audit chain are the product.

13.2 Build vs. leverage — third-party tools by layer

Build (the differentiators): orchestrator loop, atom layer + provenance, project graph, canvas, skill library, matter memory. **Leverage** everything commodity or certification-heavy:

Layer	Leverage	Why / notes
LLM + agent framework	Anthropic Claude API + Agent SDK; Claude Code as the build environment	Tool use, long context for full-document reads; keep a model-abstraction shim so models can be swapped per role (verifier ≠ drafter model is allowed).
Workflow engine	Temporal (durable execution for the Task	Long-running, crash-safe workflows; retries and

Layer	Leverage	Why / notes
	DAG)	timers (clocks!) are native. Alternative: DB-backed queue for v1, Temporal at scale.
Agent observability/evals	Langfuse (self-hostable) or LangSmith	Trace every worker run, cost, and output; eval suites per skill (golden fixtures, §11) run in CI. Non-optional once workers draft regulated content.
Document rendering/PDF	Apryse (PDFTron) SDK or Aspose	Renditions, bookmark/hyperlink control, PDF/A — the §10.1 publisher's hands. Don't hand-roll PDF.
eCTD validation	LORENZ eValidator (the reference validator, also used by agencies)	Run it as the §10.1 validation gate instead of re-implementing FDA criteria; GlobalSubmit VALIDATE is the alternative.
eCTD publishing (early)	Partner first: EXTEDO eCTDmanager or a regulatory-ops service for sequence publishing while volume is low	Build the internal publisher (§10.1) only when sequence volume justifies it; the atom/node layer already emits everything a publisher needs.
FDA transmission	A submission service provider / ESG account via partner initially; Axway or IBM Sterling AS2 if insourcing ESG later	ESG onboarding (certificates, AS2, test submissions) is slow and compliance-heavy; CDER Direct covers some submission types sooner. Anna prepares and tracks (§10.2) regardless of who

Layer	Leverage	Why / notes
		transmits.
CDISC validation	Pinnacle 21 (Community for dev, Enterprise for production)	The industry-standard SDTM/ADaM/Define-XML validator — §5 dataset intake calls it, never re-implements it.
Stats compute	R + Python containers, pinned lockfiles; SAS via licensed runtime only if a client requires	Double-programming QC (§5) is two independent container runs + a diff service (build the differ, it's small).
Terminology/ dictionaries	MedDRA + WHODrug (licensed, MSSO/UMC), SNOMED CT, LOINC + UCUM (open)	Dictionary versions are project metadata; upversioning is a scheduled watch task with impact diff on coded atoms.
FHIR/lab interop	HAPI FHIR (open-source server/library)	§7/§12.3 intake and any EHR-adjacent exchange.
OCR / document extraction	AWS Textract or Azure Document Intelligence	Data-room ingest for scanned legacy documents (§3 Evidence Librarian); extraction output still passes atom verification — OCR is a source, never a truth.
E-signature	DocuSign or Adobe Acrobat Sign (Part 11-configured offerings)	Integrate for the §8 signing ceremonies rather than building signature crypto/compliance from scratch; signature events mirror into the audit chain.
Identity/security	Okta or Entra ID (SSO/SAML, MFA), AWS	§13.1 connector credentials live here; SOC

Layer	Leverage	Why / notes
	KMS / cloud HSM for keys, vault service (HashiCorp Vault or cloud secrets manager)	2 program runs on the cloud provider's compliance inheritance (AWS/Azure) — attest the app layer, inherit the infra layer.
Search/retrieval	Postgres + pgvector first; OpenSearch when corpus scale demands	Atom retrieval (§3) starts in-database; avoid a second search stack until measured need.
Structured product data (IDMP)	EMA SPOR services + ISO IDMP data model as schema reference	§12.2 alignment; consume SPOR reference data rather than curating substance/organisation lists in-house.

13.3 Leverage sequencing

- **v1 (prove the spine):** Claude API/Agent SDK, Temporal or DB queue, pgvector, Apryse, Pinnacle 21 Community, LORENZ eValidator, DocuSign, Textract/Azure DI, Langfuse. Publishing + transmission via partner.
- **v2 (scale what clients pay for):** internal eCTD publisher, Enterprise Pinnacle 21, OpenSearch, dictionary automation.
- **v3 (insource the gateway):** own ESG/AS2 (Axway/Sterling) when sequence volume makes partner fees the bigger number.

13.4 One-paragraph brief for Claude Code

"Build the §11 phase order. Anna is a runtime: matter memory, controlled skill library, subagent spawner, connector vault, scheduler, approval gates (§13.1). Call third-party services at the seams listed in §13.2 — never re-implement validation (LORENZ, Pinnacle 21), PDF (Apryse), signatures (DocuSign), or terminology (MedDRA/WHODrug/LOINC). Every acceptance test in §§5–10 must pass against the fixture projects before the next phase starts."

13.5 Decision rights — models propose, Anna decides (v0.3, per founder direction Nov 5, 2026)

Founder's standing architecture rule: **LLMs (today: Claude and OpenAI) are cognition for hire; every real decision lives inside Anna.**

- **Workers — any model, interchangeable** — may: retrieve, draft, extract, classify, compute via tools, and **propose** (a draft, a coding, a classification, a response plan). A worker's output is a proposal until Anna accepts it. Model choice per role is a config decision (cost/quality), recorded on the Task.
- **Anna — the decision layer** — disposes inside her envelope: accept/reject/rework worker output, sequence work, code the requirements matrix, start clocks, assign response items, lock sections that pass gates. Every Anna decision is written to the audit log **with its basis** (atoms, findings, rule invoked) — decisions are data, not vibes.
- **Humans — outside the envelope** — regulatory strategy and reporting-category calls (§9.2), signatures, transmissions, exceptions to gates. Anna prepares these as decision packages; she never takes them.
- **Enforcement is structural:** worker API tokens cannot lock, sign, transmit, or alter clocks (§13.1 approval gates). Swapping Claude for OpenAI — or adding a third model — changes no decision rights, because the rights were never the model's.

---

14. Can AnA herself live inside the product? — feasibility investigation (v0.4)

The founder's question, stated plainly: *the assistant he works with every day (AnA) runs agent orchestration well — can she be put inside Concept to Cure itself?*

14.1 The honest answer

AnA cannot be lifted out and installed. The AnA the founder experiences is an agent runtime running on Meta's Muse platform. Her working parts are platform machinery, not a portable component: persistent memory files, a skill library, subagent orchestration, a connector vault holding scoped credentials, scheduled watchers, and runtime approval gates. There is no package to copy, no binary to embed, and no supported path to extract that runtime from its host platform.

What can be built — and what this spec designs — is Anna as a product runtime inside Concept to Cure that implements the same architecture. §13.1's six components (persistent matter memory, controlled skill library, subagent spawner, tool & connector layer, scheduler/watchers, approval gates) are a design, and a design can be built anywhere. The product Anna runs on the Claude API / Agent SDK (with OpenAI where chosen per role, §13.5), holds the same decision rights (§13.5), and keeps the same audit discipline (§8). From the user's chair the behavior is the same: one voice, matter memory, parallel workers, gates that hold. Under the hood she is a service in the founder's codebase — not a guest from another platform. That distinction matters commercially (no dependency on a third party's assistant platform) and for validation (the product Anna's code, prompts, and decisions are all inside the platform's own QMS, §9).

14.2 Three implementation options

Option	What it is	Verdict
A. Native Anna runtime service	The full §13.1 runtime built into the v2 codebase: orchestrator loop, matter memory store, controlled skill library, subagent spawner over the Agent SDK, connector vault, scheduler, approval gates as platform objects	**Recommended for production.** The only option where §13.5 decision rights live inside the product — Anna-the-service accepts, rejects, sequences, and locks; models only propose. Everything else in this spec (§§2–12) already assumes this shape.
B. MCP tool layer	The platform's functions (`dataroom`, `atoms`, `draft`, `compute`, `render`, `findings` — the §3 tool contracts) exposed as MCP tools, driven by an agent host running outside the product	**Spike/prototype path only.** Fastest way to learn what orchestration feels like against the real data layer, and the tool contracts get built either way. But the decision layer sits outside the product, which violates the §13.5 principle for production: whoever hosts the agent holds the decisions. Use it to learn;

Option	What it is	Verdict
		do not ship it as Anna.
C. Embedded copilot only	Chat + drafting assistance inside the canvas (§8): answers questions, drafts from atoms on request. No orchestration, no subagents, no memory across matters	**Fallback, and name it honestly.** A useful feature, but it is not Anna — no task ledger, no gates of her own, no ledger discipline. Ship it under a feature name if A proves too heavy for v1; never market it as the orchestrator.

14.3 Codebase audit task (for Claude Code — do this first)

The live codebase is **ClinicalSageAI-2-replit, branch `concept2cure-v2`**. From the Nov 5, 2026 working session it already contains: a Vault/Data Room, lineage tracking, eCTD/RIM work, AnA drafting, four connected connectors, and multi-lane build coordination via `COORDINATION.md`. Some of §13.1 may therefore already exist in embryo. Before any plan is written, audit the branch against §13.1's six components:

§13.1 component	Audit question
Persistent matter memory	Is there a per-client/per-project store of facts, decisions, and standing rules that survives sessions and is loaded at session start? Or only conversation state?
Skill library	Are there versioned procedure/playbook artifacts (prompt files, workflows) with tool grants and gates — and are changes to them controlled and logged?
Subagent spawner	Can the system dispatch a stateless worker with a self-contained brief and input bundle, run workers in parallel with bounded concurrency, and return results to a single merger?
Tool & connector layer	Do the four connectors sit behind

§13.1 component	Audit question
	scoped, task-level tokens from a vault — or do callers hold raw credentials? Do the §3 tool contracts (or equivalents) exist as APIs?
Scheduler / watchers	Are there cron-like jobs and event hooks with owners and stop conditions (clock checks, ACK watches), or only request-driven execution?
Approval gates	Are signature / transmission / judgment pauses enforced by platform objects a worker cannot bypass — or by prompt instruction alone?

Audit output format, per component: status (**EXISTS / PARTIAL / MISSING**), file-level evidence (paths, not impressions), and the **smallest adapter** that turns what exists into the component as §13.1 specifies it. Where the codebase's existing pattern is better than this spec's assumption, say so and name the spec section that should change — the audit corrects the spec, not the other way round.

14.4 Spike spec (time-boxed: 2 weeks)

One fixture project (reuse a §11 golden fixture). The product Anna, running as a service in the v2 codebase (Option A shape; an Option B harness is acceptable for the spike only if the gate and audit criteria below are still enforced inside the product):

1. Anna accepts a typed or spoken instruction from the user ("draft section X from the verified atoms and check it").
2. She composes Tasks and **spawns two workers through the spawner**: a Section Drafter and a Citation Verifier (drafter ≠ checker, §1 principle 3).
3. She routes the draft to verification, merges only verified work, and attempts a section lock — the lock is an **approval gate**: a worker holding only its task-scoped token tries and **cannot** perform it; the lock completes only on Anna's disposition inside her envelope (§13.5), recorded as an audit event.
4. **Every Anna decision** in the run (accept, reject, rework, sequence, lock) is written to the audit log **with its basis** — atom refs, finding refs, rule invoked (§13.5).

Pass: all four steps run end-to-end on the fixture; the worker-token lock attempt fails and the failure is itself audit-logged; the decision log reconstructs the whole run without reading conversation text. **Fail:** any step requires a human to hand-carry state between components, the gate is enforced by prompt wording rather than by token scope, or a decision lands in the log without a basis. A fail is a finding, not a disaster — it prices Option A honestly.

14.5 Risks and open questions

- **Part 11 / validation posture for agent-made decisions.** A regulated customer will ask who decided, on what evidence, under which controlled procedure. The mitigation is already the design: decisions are audit-logged with basis (§13.5), skills are controlled documents (§9.2), locks and signatures are human-gated (§8). Open: whether Anna's *internal* dispositions (accept/reject/rework) need the same validation package as user-facing approvals in a customer's IQ/OQ — scope this with the first design-partner, don't guess.
- **Per-matter tenant isolation of memory.** Matter memory is the moat (§13.1) and the liability: one client's facts must be structurally unreachable from another client's sessions. Memory stores are per-tenant with no cross-tenant retrieval path, enforced at the data layer — the atom jurisdiction tags (§2.3) are the model; extend the same discipline to memory records.
- **Model cost and latency budgets per document type.** Orchestration multiplies model calls (draft + verify + red-team per section). Set a budget per document type (CSR section, Module 3 table set, CER) as a product requirement: measured on the spike and the §11 fixtures, enforced by the spawner's concurrency and model-choice config (§13.5 records model per Task).
- **Evaluation harness.** Anna's quality is only as real as her evals: the §11 golden fixture projects are the harness — every skill change (§9 change control) re-runs them, with Langfuse/LangSmith traces (§13.2) as evidence. No fixture coverage for a document type = that type does not ship with Anna enabled.
- **Founder decisions still open:** (1) Option A for v1, or Option C shipped first with A on the roadmap; (2) whether Anna is **customer-facing** (the client's users talk to her) or **internal-only** (the platform's operators talk to her; clients see outputs) at first — this changes the memory-isolation and validation scope materially; (3) whether the Option B MCP layer is built anyway as the integration surface for client-side agents, separate from the Anna question.

14.6 What this investigation delivers

A short audit report (§14.3) plus the spike result (§14.4) — nothing more, nothing vaguer. Together they price Option A against the real codebase instead of against this spec's assumptions, and they give the founder a clean **go / no-go on Option A for the roadmap**, with Option C scoped as the fallback if the answer is no-go for now.

---

15. Handoff brief for Claude Code (roadmap consideration) (v0.4)

Self-contained cover brief — paste this section to Claude Code together with the full spec.

```
CLAUDE CODE — ROADMAP CONSIDERATION BRIEF: THE ANNA RUNTIME
Concept to Cure · attached: Multi-Agent Architecture Spec v0.4

WHAT THIS DOCUMENT IS
The build specification for Concept to Cure's multi-agent
architecture:
an orchestrator (Anna) holding the project graph, the rules, and the
memory; stateless specialist workers executing against a provenance-
first
atom layer; deterministic code computing every number; verification
gates
on everything. §§1–12 are the platform. §13 is Anna implemented as a
runtime (six components, §13.1) plus the build-vs-leverage map
(§13.2).
§13.5 fixes decision rights: models propose, Anna decides, humans
sign.
§14 investigates whether the founder's own assistant (AnA, running on
an
external platform) can live inside this product — the short answer is
in
§14.1 — and tasks the codebase audit and spike that settle it.

WHAT ALREADY EXISTS (fill this in from the audit, §14.3)
The live codebase is ClinicalSageAI-2-replit, branch concept2cure-v2,
with — as of Nov 5, 2026 — a Vault/Data Room, lineage tracking,
eCTD/RIM
work, AnA drafting, four connectors, and multi-lane coordination via
COORDINATION.md. State here, per §13.1 component, what the audit
found:
EXISTS / PARTIAL / MISSING, with file evidence and the smallest
adapter.
```

```
THE ASK — CONSIDER AND PLAN, DO NOT BUILD YET
Produce four things for the founder to approve:
1. The §14.3 codebase audit (format as specified there).
2. A phased implementation plan mapped onto §11's build order, showing
   exactly where the Anna runtime lands — which §11 phase each §13.1
   component arrives in, and what changes in phases already underway.
3. The §14.4 spike: time-boxed at 2 weeks, pass/fail as written there.
   (The spike is the one piece of building authorized by this brief.)
4. A roadmap entry: one epic ("Anna runtime") with milestones tied to
   the §11 phases and the §14.6 go/no-go, written for the founder to
   approve, defer, or cut — including the Option C fallback (§14.2).

GUARDRAILS (non-negotiable)
- §13.5 decision rights: workers — any model — propose only. They
never
  lock, sign, transmit, or alter clocks; enforcement is structural
  (token scope), never prompt wording.
- Third-party seams per §13.2: never re-implement validation (LORENZ /
  Pinnacle 21), PDF (Apryse), signatures (DocuSign), or terminology
  (MedDRA / WHODrug / LOINC). Build the differentiators: orchestrator,
  atom layer, project graph, canvas, skill library, matter memory.
- Every phase is gated on its fixture-project acceptance tests (§§5–
11).
  No phase starts on a red gate.
- If the codebase already does something better than this spec
assumes,
  flag the spec section that should change. The audit corrects the
spec.
```

---

16. Trust architecture — the design source: how AnA handles secrets (v0.5 addition)

§13.1 names a connector vault and approval gates as Anna components, and §10.2 says gateway credentials “live in a vault, never with workers.” This part specifies the trust model behind those lines, because for regulated clients it is not an implementation detail — it is the buying decision.

The design source is factual: the operating pattern of AnA, the assistant runtime the founder works with daily (§14). That pattern is worth naming precisely because Concept to Cure productizes it: **the client keeps custody of every account, every credential, and every byte; Anna is a revocable agent who works inside the client’s walls, on scoped grants, with consequential actions approved by a human and every action logged.** Custody never transfers. What transfers is work.

16.1 The AnA pattern — five mechanisms

1. Secure Vault credential cards. When a password or key is needed, the human enters it on a secure capture surface provided by the platform. The agent never receives the raw value — not in chat, not in context, not in a log. At the moment of use, the runtime presents the credential to the target system itself. The agent learns only whether it worked. An agent cannot disclose what it never held: the secret is unrepresentable in agent context, which is a data-flow fact, not a policy promise.

2. Scoped connectors. Every connected account is an OAuth-style grant: least-privilege scopes, tied to that account, revocable by the human at any time. A connection grants capability for tasks — it does not grant blanket permission, and access to one account implies nothing about any other account.

3. Approval cards. Consequential actions — sending, paying, submitting, attesting — pause at a runtime gate. The human approves the *exact* action and content shown. Approval covers what was shown and nothing more; change the content and the approval no longer applies. No approval, no action — enforced by the runtime, not by the agent's politeness.

4. Discretion. The agent uses only what the task needs. Private context does not leak into unrelated queries, messages, or outputs; each outside party receives only the details they need to act. Minimum-necessary is the default data posture, in both directions.

5. Decision rights + audit. Workers propose, Anna disposes inside a defined envelope, humans decide outside it (§13.5) — and both the dispositions and the approvals land in an audit log with their basis. "Who decided this, on what evidence, under which rule" is always answerable from the log.

Mechanism	Enforced by	Failure mode it removes
Vault credential cards	Runtime / secrets layer	Agent (or its logs, or its model provider) ever holding a client secret
Scoped connectors	Connector grants, token scopes	Blanket access; one account's grant leaking into another's
Approval cards	Runtime gates (§13.1)	Anything consequential leaving on an agent's initiative

Mechanism	Enforced by	Failure mode it removes
Discretion	Context assembly (minimum-necessary bundles, §3)	Cross-matter and cross-party leakage
Decision rights + audit	§13.5 + §8 audit chain	Untraceable decisions; “the AI did it” as an answer

17. The product mapping — custody stays with the client (v0.5 addition)

Each AnA mechanism above becomes a product feature. The section references are deliberate: this part extends §§8, 10, 12, and 13 rather than restating them.

17.1 Per-client tenant workspaces — hard isolation

```
Tenant { id, client_org, status: "onboarding" | "active" |
"offboarding" | "closed",
         isolation: { data_partition, encryption_key_ref,
memory_partition },
         data_classes: ["submission_data", "cmc_trade_secret",
"regulatory_correspondence", ...] }
```

- Every Vault object (§12.1), atom (§2.3), memory record (§13.1), and audit event carries a `tenant_id`; retrieval paths are tenant-scoped at the data layer, extending §14.5’s memory-isolation rule to all data classes. There is no cross-tenant query path — not for workers, not for Anna, and not for platform support staff, whose access is itself a gated, logged approval event (§17.6 pattern).
- A sponsor’s submission data, CMC trade secrets, and regulatory correspondence are therefore never co-mingled, never used to serve another client, and never visible in another tenant’s sessions.

17.2 Client-owned authority accounts — the core rule

```
AuthorityAccount { id, tenant_id,
                   authority: "FDA-ESG" | "FDA-CDRH-Portal" | "CDER-
Direct" | "EMA-IRIS"
                            | "MHRA" | "HC-CESG" |
"ClinicalTrials.gov-PRS" | ...,
                   account_owner: "client",          // invariant —
never "c2c"
                   account_ref,                       // identifier
```

```
only; no secrets here
                    status }
CredentialGrant  { id, authority_account_id, granted_by,   // a client
admin (§17.8)
                    scope: ["transmit_sequence", "fetch_acks",
"read_correspondence", ...],
                    vault_ref, expires_at?, revoked_at?, last_used_at }
```

Rule: Concept to Cure holds no house accounts for client filings. Every submission issues from an account the client owns; Anna acts as the client's preparer/agent under that account and under a scoped CredentialGrant. This is the product form of §16.1's mechanisms 1–2: the vault holds the client's credential, the grant scopes what Anna may do with it, and the client can rotate or revoke at will (§17.5).

17.3 Drugs and biologics — FDA ESG, under the sponsor's account

eCTD submissions (IND/NDA/BLA and their sequences, §10) transmit to FDA through the Electronic Submissions Gateway (ESG). In this architecture the Submission's `gateway` object (§10.2) references the sponsor's own AuthorityAccount, and transmission executes under that account — Anna prepares, the validation gate (§10.1) and the approval gate (§17.6) clear the sequence, and the transmission service (§17.5) submits as the sponsor's agent.

Why sponsors insist on their own account — the reasons are structural, and the product states them plainly:

- **Regulatory ownership.** The application belongs to the sponsor, and the sponsor remains responsible to FDA for it regardless of who prepared the filing. The account of record should match the owner of record.
- **Receipts belong to the account holder.** Gateway and center acknowledgments (§10.2's ACK chain) are delivered to the transmitting account. If a vendor's account transmits, the sponsor's proof of receipt lives in the vendor's mailbox.
- **Continuity.** Changing vendors or bringing regulatory operations in-house must not strand submission history, gateway access, or correspondence behind a former vendor's account.
- **Inspection readiness.** The sponsor can produce the complete transmission record from its own account, on demand, without asking a vendor for exports.

(Verify before external use: the exact ESG onboarding mechanics — WebTrader vs. AS2, digital-certificate issuance, and FDA's current rules for third-party transmission on a

sponsor's behalf. The product model above is the design commitment; the per-authority checklist in §17.9 is where the verified mechanics live.)

17.4 Device (MDX) clients — eSTAR, under the client's CDRH account

For device clients (§4.3), the same model applies to FDA's eSTAR: eSTAR is FDA's electronic submission template for premarket device submissions, and submissions are filed through the client's own CDRH portal account. Anna builds the eSTAR package from the 510(k) graph (§10.1 mapping), the client approves the exact final package (§17.6), and transmission executes under the client's account, with portal receipts ingested as Submission events (§10.2) and any FDA follow-up entering the Correspondence intake (§10.3).

(Verify before external use: eSTAR's current mandate scope. FDA has required 510(k) submissions to be submitted electronically using eSTAR since October 1, 2023, subject to exemptions, and eSTAR's coverage across premarket types has been expanding — confirm the current scope for each submission type before stating it to a client.)

17.5 Vault-style credential delegation inside the product

- A client admin enters ESG, CDRH portal, and other authority credentials (and certificates, where an authority uses them) into a **secure capture surface inside Concept to Cure** — the product form of the vault credential card (§16.1). Credentials are stored in the secrets layer (§13.2: cloud KMS / secrets manager), never in the application database in plaintext, and **never in any agent's context, prompt, or log**.
- At submission time, the *transmission service* — deterministic platform code, not an LLM worker — retrieves the credential under the CredentialGrant's scope and presents it to the gateway or portal. Workers and Anna handle account references and grant scopes only.
- The client can **rotate** a credential at any time (rotation is a vault event, not a support ticket) and **revoke** a grant at any time; revocation is immediate and invalidates derived sessions and tokens.
- Every credential *use* is an audit event — which grant, which action, which submission, when. The value is never part of the event.

17.6 Submission approval workflow — the approval card, productized

```
ApprovalEvent { id, tenant_id, object_ref, content_hash,
                meaning: "approve_transmission" |
"approve_publication" | ...,
                approved_by, role, approved_at, superseded_by? }
```

- No filing transmits without a client ApprovalEvent covering the **exact final package**: the approval binds to the package's content hash (§10.1 checksums make this natural). Any change to the package after approval changes the hash, voids the approval, and returns the sequence to the gate. This is §10.2's "human approval signature in QMS" and §8's meaning-of-signature made explicit and structural.
- The approval is recorded on the Submission (state `ready → transmitted` requires a valid, un-superseded ApprovalEvent) and in the audit log (§17.7). Anna prepares the approval package — validation report, sequence contents, diff against the prior sequence — so the human approves something inspectable, not a filename.

17.7 Client-visible audit log

Extends §8's hash-chained log with the trust events of this part: credential grants, rotations, revocations, and uses; ApprovalEvents; transmissions and acknowledgments; Anna's decisions with basis (§13.5); support-access events. The log is **client-visible and exportable per tenant** — it ships inside the §9.3 inspection pack.

The layer is **designed for alignment with 21 CFR Part 11 expectations** — complete audit trails, e-signature manifestation and meaning, access control, and record retention — as a design target. It is not a certification claim, and each deployment's Part 11 posture is validated in the customer's own IQ/OQ process (§14.5).

17.8 Account lifecycle and roles

Onboarding: tenant provisioning with isolation keys (§17.1) → role setup (below) → authority account registration (§17.2) → credential capture (§17.5) → grant scoping → a test transmission where the authority provides a test channel *(verify per authority before external use)*. **Roles** (segregated per §8; one person may hold several, but events record the role acted in):

Role	Holds
Tenant admin	Accounts, credentials, grants, users — the only role that touches §17.5
Regulatory lead	Owns the authority relationship; approves submissions
Author	Drafts and assembles (agent-assisted)

Role	Holds
Reviewer	Reviews, comments, raises Findings (§2.6)
Approver	Signs ApprovalEvents (§17.6); may be the regulatory lead, recorded distinctly

Offboarding: full export of the tenant's Vault content, atoms with provenance, audit log, and correspondence in open formats; credential revocation and grant termination recorded as audit events; retention or disposition of any platform-side copy follows the schedule in the customer contract and the customer's QMS — stated as a contract term, never improvised.

17.9 One vault, many authorities

The same delegation pattern extends to every account the client owns:

Authority / system	Used for
FDA ESG	eCTD submissions, drugs/biologics (§17.3)
FDA CDRH Portal	eSTAR device submissions (§17.4)
CDER Direct	Submission types FDA routes outside ESG *(verify current scope)*
EMA IRIS	EU regulatory activities
MHRA portal	UK submissions
Health Canada CESG	Canadian submissions
ClinicalTrials.gov PRS	Registration and results reporting
Client EDC / eTMF / CTMS	Scoped connectors into the client's own systems (§13.1)

Each authority's delegation and account rules differ; enabling one is a configuration plus a verified onboarding checklist (§17.8), not a code change. The invariant does not change: **the account owner is always the client.**

18. Why this wins (v0.5 addition)

The number-one objection regulated clients raise to AI agents is custody: *who holds our credentials, our submission accounts, our data — and what happens to all of*

it if we leave? Contract language answers that weakly; architecture answers it completely. Here the client keeps ownership of every account (§17.2) and every byte (§17.1), Anna operates as a revocable agent inside their walls (§§17.5–17.6), and the audit trail (§17.7) makes every action inspectable by the client, not just by the vendor. It is the same principle as §13.5, one layer down: models propose, Anna disposes, humans sign — and the platform operates, the client owns, humans approve. A competitor who transmits from house accounts and holds client data in a shared store cannot match this with a policy page; it has to be built this way.

---

19. Build implications — trust architecture (v0.5 addition)

For the §14.4 spike, prototype these three first — they are the trust architecture in miniature:

1. **Vault capture surface + CredentialGrant:** a client admin enters a (fixture) credential; the transmission service retrieves it under grant scope at use time. *Test:* a canary secret planted via the capture surface appears nowhere in agent context, traces, or logs — grep the run's artifacts for it; the only permitted record is the credential-use audit event, which contains no value.
2. **Approval flow bound to content hash (§17.6):** approve a fixture package, then change one byte. *Test:* the approval is void, transmission is blocked, and both events are in the audit log.
3. **Audit log schema** covering decision (§13.5), credential-use, approval, and transmission events, tenant-scoped. *Test:* a tenant-A token querying tenant-B audit records returns nothing.

Later phases (§11): authority onboarding checklists (§17.8) land with phase 7 (publishing/submission); support-access gating and the offboarding export (§17.8) land with the inspection pack (§9.3).

---

20. Continuous evolution — the design source: how AnA learns (v0.5 addition)

§13.1's matter memory stores what Anna *knows*. This part covers what Anna *learns about how to work*: the rules. The moat claim in §13.1 — memory + skills + gates —

only compounds if the rule set grows from experience instead of being re-taught every session. The design source is again AnA's actual operating pattern.

20.1 Persistent memory across sessions

AnA carries durable records across sessions: preferences, decisions, the people involved, commitments made, corrections received. The user never re-explains what was settled last week; each session starts from the accumulated record, and durable learnings are written back as they occur. Memory records are typed (preference / decision / person / commitment / correction), carry their source, and are corrected in place when the record proves wrong — a corrected memory names what it replaced.

20.2 The correction → rule pipeline

When the human corrects Anna — or is visibly frustrated with an output — the response follows a fixed pipeline:

1. **Fix the instance.** The immediate work product is corrected first.
2. **Extract the general rule** behind the correction — the class of situation, not just this case.
3. **Save it as a standing rule**, with the correction event as its source.
4. **Apply it prospectively** everywhere it fits, without being asked again.
5. **Own the miss in one sentence, state the fix, move on.** No excuse paragraphs; the repair is behavioral, not rhetorical.

Pattern examples, stated generically because the shape recurs: a rejected document format became a permanent template rule (never present work in any other form); a message sent through the wrong channel became a hard channel rule (each class of work has exactly one authoritative channel); a phrasing complaint became a standing voice rule (a register the user would actually write in, applied to all outbound text).

20.3 Guardrail escalation — preferences harden

Rules are not all born equal; they escalate:

Level	Form	Behavior
L1 — soft preference	A noted leaning	Default behavior; the human can override it in the moment without ceremony

Level	Form	Behavior
L2 — standing rule	Saved rule with source (§20.2)	Applied everywhere it fits; exceptions are the human's to make, explicitly
L3 — hard gate	Runtime block (§13.1 approval gates)	The action cannot proceed without explicit human approval — sends, submissions, payments, attestations

Promotion triggers: a repeated correction (the same miss twice is a system failure, not bad luck), severity (a miss that could embarrass or harm promotes immediately), or a direct human instruction ("never again" promotes on the spot). Demotion is the human's alone — Anna never quietly relaxes a rule that a correction created.

20.4 Alignment synthesis — periodic self-review

Periodically, AnA re-derives the working relationship from the record: the current rule set, how the user works, open frictions, and repair threads still outstanding. The point is drift detection — behavior is checked against the rule ledger, divergences are named, and stale rules surface for the human to keep, amend, or retire. The relationship is maintained like a controlled document, not left to accumulate.

20.5 Trust repair

When trust ruptures, the repair is tracked openly as work: acknowledge what happened, correct the behavior (usually via §20.2 — the rupture produces a rule), and confirm the repair landed. A repair thread stays open until the human's experience confirms it; it is never closed by declaration and never papered over.

---

21. The product mapping — memory and rules at three scopes (v0.5 addition)

In the product, everything in §20 is scoped. A rule learned in one scope must never leak into another — learned rules get exactly the isolation that §17.1 gives data.

```
RuleRecord { id, scope: "user" | "tenant" | "project", scope_ref,
             rule, source_event_ref,            // the correction /
onboarding decision that created it
             level: "soft" | "standing" | "hard_gate",
             state: "active" | "revoked" | "superseded", created_at,
revoked_by? }
```

21.1 The three scopes

Scope	Holds	Examples
Per human user	Personal working style	Voice and formatting rules, correction history, approval habits — Anna adapts to the individual regulatory writer or lead: how they like drafts marked up, what they always check first, what never to send without their eyes
Per customer (tenant)	Organizational rules	House templates, approved terminology, sign-off chains, regulatory posture, which authorities the customer files with (§17.2). Owned by the customer, exportable with the tenant's data (§17.8), and **never shared across tenants**
Per project (submission)	Matter-specific decisions	Precedent choices for this application, positions taken in prior sequences, agency feedback on this application (§10.3 correspondence) and the rules that follow from it — e.g., a reviewer's stated preference becomes a project rule

Scope	Holds	Examples
		for every later sequence

21.2 Precedence and conflicts

More specific scopes win: project rules override tenant rules, which override user rules, *for the matter at hand*. Two exceptions are structural: a **hard gate at any scope overrides softer rules at every scope** (a tenant's "nothing transmits without the regulatory lead" gate is not relaxed by a user's preference), and a genuine conflict between same-level rules is never silently resolved — Anna surfaces it as a decision package (§13.5) and the resolution becomes a rule itself.

21.3 The "Rules Anna follows" ledger

Every scope has a user-visible ledger: the rules Anna is currently applying, each shown with its source event and date. Users and tenant admins can **inspect, edit, and revoke** rules in their scope; revocation and editing are audit events (§17.7). Nothing Anna does rests on a rule the customer cannot see — the ledger is the answer to "why does she keep doing it that way," and the control surface when the answer is wrong.

21.4 Alignment digest

The §20.4 synthesis, productized: a periodic digest per user (rules learned, frictions open) and per tenant admin (rules learned at tenant scope, guardrail promotions and changes, repair threads and their status). The digest is delivered as a platform communication (§12.5) with a WorkTask (§12.4) for any item needing a decision, so guardrail changes are reviewed like everything else in the platform — on the record.

21.5 Isolation of learning

Memory records and RuleRecords carry `tenant_id` and are retrieved only inside their scope (§17.1, §14.5). One customer's house style, posture, and corrections can never surface in another customer's sessions. Platform-level improvement travels by a different road: as new versions of the controlled skill library and templates, shipped under QMS change control (§9, §13.1) — product updates, not tenant knowledge in transit.

21.6 Trust repair in the product

§20.5 becomes an incident workflow: a platform-caused failure (a mis-routed document, a wrong-scope rule applied) opens a record linked to the QMS Deviation/CAPA machinery (§9.1) where the cause is the platform, with a client-visible status and a fix + rule-change entry in the tenant's ledger (§21.3). The thread closes on the client's confirmation, not on the platform's say-so.

---

22. Why this wins (v0.5 addition)

Software is re-bought; accumulated understanding is not. A competitor can match a feature list in a year; it cannot match two years of a customer's own rules, precedents, and corrections — held inside the customer's own walls (§17), visible in their ledger (§21.3), and exportable with their data if they ever leave. The switching cost Anna creates is *earned familiarity*, not hostage-taking — the same posture as client-owned authority accounts (§17.2), applied to knowledge. And because the rules are explicit records rather than model weights, the familiarity survives model swaps (§13.5): change the underlying model and Anna still works the way the customer taught her.

---

23. Build implications — continuous evolution (v0.5 addition)

For the §14.4 spike, add one prototype to the trust items in §19:

1. **Correction capture + RuleRecord:** on the fixture tenant, a user correction in-session produces a RuleRecord with its source event, visible in a read-only ledger view; the rule demonstrably changes Anna's behavior on the next task of the same class.
2. **Escalation stub:** marking a rule "never again" promotes it to a hard gate wired to the §19 approval flow — the gated action then blocks without approval.

Later phases: the precedence/conflict resolver (with fixture tests per §21.2), the alignment digest generator (§21.4), per-project rule derivation from correspondence (§10.3 classifier proposes, Anna disposes, §13.5), and ledger edit/revoke with full audit wiring (§17.7).

---

24. Data locality — Anna's routing doctrine (v0.5 addition)

The dilemma, stated honestly: much of a client's data must never go to a frontier large language model — and frontier models are, for some work, genuinely needed. Most vendors resolve this by picking a side. This architecture resolves it by **routing**: classification-driven, three lanes, orchestrated by Anna under a customer-set policy. The doctrine, in one line: **raw data stays home; reasoning travels on derivatives.**

24.1 Classify first — sensitivity tiers

Every document and data element is auto-tagged at ingest with a sensitivity tier (a new field on atoms, §2.3, and on VaultDocuments, §12.1):

Tier	Class	Examples
Tier 0	Public	Published guidance, public labels, literature
Tier 1	Internal	Project plans, general correspondence, non-sensitive drafts
Tier 2	Client-confidential	Client documents, study data, submission drafts
Tier 3	Restricted	Trade secrets (CMC know-how), unblinded data, pre-decisional submission content, personal data

Classification is performed by the local lane (§24.2, Lane A) at ingest — classification itself never requires sending content out. Default tiers attach to data classes; the customer refines them per tenant and per project through the rule system (§21) — a tier assignment is a RuleRecord, visible in the ledger (§21.3). When classification is uncertain, the element tiers **up**, not down: doubt resolves toward the more protective lane.

24.2 Three lanes

```
RoutingDecision { id, tenant_id, task_ref, tier_max,          //
highest tier in the input bundle
                  lane: "A_local" | "B_frontier" | "C_derivative",
                  model_ref, derivative_ref?, policy_ref, at }  //
every decision is an audit event (§17.7)
```

- **Lane A — Local only.** Models hosted inside the client's tenant/VPC (open-weight or locally hosted models) do all work that needs raw Tier 2/3 content: extraction, parsing, classification, QC checks, citation verification, formatting. Raw restricted content never leaves the tenant. Note what this preserves from §1: the deterministic-vs-LLM split and drafter ≠ checker both hold inside Lane A — verification of restricted content is local work.
- **Lane B — Frontier, zero-retention.** Frontier models, reached through enterprise APIs under contractual no-training / no-retention terms, handle work needing top-end reasoning over Tier 0/1 content — and Tier 2 content only where the customer's policy (§24.3) expressly allows it. *(Verify before external use: each provider's current enterprise terms — retention windows, training use, regional processing — before a provider is enabled for any lane.)*
- **Lane C — Frontier on derivatives only.** For restricted matters, the local layer first produces a **sanitized derivative**:

```
Derivative { id, tenant_id, source_refs: [],                    // stays
in-tenant; refs never leave
             transforms: ["entity_tokenize", "generalize",
"excerpt_minimize"],
             token_map_vault_ref,                                 // the
token→entity map lives in a vault INSIDE the tenant
             tier_after: "0" | "1", created_by: "lane_a_service", at }
```

Names, molecules, values, and identifiers are replaced by tokens whose mapping exists only inside the tenant and is reversible only there; content is generalized and excerpt-minimized to what the reasoning task actually needs. Anna then **decomposes the task**: the frontier model plans, argues, and drafts language over the derivative; local models inject the restricted specifics inside the tenant; a local merge step assembles the final document; verification runs locally. The frontier model sees the *shape* of the problem — the argument structure, the regulatory posture, the drafting problem — never the client's secrets.

24.3 The policy engine — the customer sets the ceiling

```
TenantDataPolicy { tenant_id, ceiling_by_tier: { "T3": "A", "T2": "C",
"T1": "B", "T0": "B" },
                   residency: [], approved_providers: [],
                   posture: "full_local" | "hybrid" |
"frontier_forward", version, approved_by }
```

- The customer sets the maximum lane per tier ("nothing Tier 2 or above leaves this tenant") plus residency constraints and the approved provider list. Anna enforces the policy on **every** model call: a Task's routing is computed from its input bundle's highest tier and the policy ceiling, and a

call that would exceed the ceiling does not happen — the task re-routes, decomposes (§24.2 Lane C), or escalates.
- Posture is selectable per client — **full-local** (Lane A only), **hybrid** (the default above), **frontier-forward** (Lane B wherever tiers allow) — and changeable; a policy change is a tenant-admin approval event (§17.6 pattern) under change control (§9).
- Every outbound call is written to the audit trail (§17.7) with the data class it carried and the RoutingDecision that authorized it. "What has ever left this tenant, in what form, under which policy version" is a report, not an investigation.

24.4 Honest trade-offs

- **Where local models suffice today:** extraction, parsing, classification, citation verification, QC and consistency checks, formatting and template work — high-volume, well-bounded task classes with verifiable outputs (§1's born-verified and recompute checks suit them exactly).
- **Where frontier strength is genuinely needed:** novel argumentation, cross-domain reasoning (a CMC fact pattern meeting a clinical claim), strategy drafting, and the highest-quality long-form drafting language. Lane C exists precisely so this strength is available on restricted matters without the content traveling.
- **The costs are real:** hybrid routing adds latency (derivative production, local merge), infrastructure cost (hosting local models per tenant), and a capability ceiling on purely local work. The spec does not pretend otherwise to the customer — the policy engine (§24.3) is where each customer prices the trade-off for themselves.
- **Evaluation gates discipline the lanes:** a task class routes local-only or derivative-only only after it passes accuracy benchmarks on **the customer's own historical documents** (the §11 golden-fixture discipline, per tenant — fixtures stay inside the tenant). Until a task class passes, it routes to the stronger lane the policy allows, or escalates. Routing is earned per task class, per tenant — never assumed from a vendor benchmark.

---

25. Why this wins (v0.5 addition)

Competitors offer regulated clients a choice between two failures: "send us everything" (frontier AI on raw data, custody surrendered — the objection §18 answers) or local-only AI too weak for the reasoning work that matters. The

routing doctrine refuses the choice: the client's secrets never leave, and they still get frontier-grade thinking on everything that can be safely abstracted — which, once Anna decomposes the work, is most of it. Combined with §§16–17, the offer is coherent end to end: your accounts, your credentials, your data home — and the strongest available reasoning, applied where it is safe.

---

26. Build implications — data locality (v0.5 addition)

For the §14.4 spike, add two prototypes to §19 and §23:

1. **Tier tagging + routing stub:** ingest on the fixture tenant auto-tags tiers; the spawner computes a RoutingDecision per Task from bundle tier + a fixture TenantDataPolicy, and the decision lands in the audit log. Test: a Task whose bundle contains a Tier 3 atom cannot be routed to a frontier endpoint — the attempt is blocked and logged.
2. **Derivative round-trip:** the local layer tokenizes a fixture document into a Derivative; a frontier call drafts against the derivative; the local merge re-injects specifics. Test: the outbound payload contains no fixture secret strings (grep the captured payload for the seeded values — they must be absent; the tokens must be present), and the merged document restores them exactly.

Later phases: per-tenant local-model hosting and the evaluation-gate harness (§24.4), provider-terms verification checklist per enabled provider (§24.2), and residency enforcement (§24.3).

---

27. Three kinds of learning — what crosses, what never does (v0.5 addition)

§21.5 isolated learning inside tenants. Left alone, that reads as a limit: an Anna who can never learn from one customer for another's benefit. The resolution is to separate **three kinds of learning**, each with its own wall — because they are different substances and fail differently when they leak.

27.1 Kind 1 — Client content: the absolute wall

Facts, data, documents, tenant rules (§21), strategies, positions, identities. **Never crosses tenants** — not in raw form, not in summaries, not in anyone's session

context. This is the invariant established in §17.1 and §21.5, restated here because everything below is an exception-shaped thing that must never be mistaken for an exception to it: Kinds 2 and 3 move *patterns and craft*, and neither is a channel for content.

27.2 Kind 2 — De-identified patterns: crosses aggregated, through a gate

Aggregate, anonymized experience: that reviewers push back on a particular briefing structure; that a deficiency class recurs across applications; typical cycle times for a review path. This is the industry's collective scar tissue, and it is valuable precisely because no single customer sees enough of it.

- Patterns cross **only in aggregated form, through a de-identification gate run by the local layer** — the same tokenization machinery as Lane C derivatives (§24.2), pointed at experience instead of documents: no client identity, no client facts, no values that identify a program, and no reversible path back (the mapping, if any exists, never leaves the tenant).
- A pattern is promotable only when it is supported across a **cohort of tenants** (minimum cohort size is a platform policy parameter), so no single customer's experience can be reverse-engineered from the "aggregate."
- **Customers can opt out** of contributing even at this level (a TenantDataPolicy setting, §24.3). Opting out costs them nothing in service — they still *receive* the benefit of Kind 3 craft (§27.3) — it only stops their tenant from contributing pattern signals.
- The audit trail (§17.7) shows each tenant **what classes of pattern-learning left their tenant** — classes and counts, never content — so contribution is inspectable, not a matter of faith.

27.3 Kind 3 — Anna's craft: compounds globally

Anna's own skills and playbooks: better extraction methods, sharper routing judgment (§24), improved generic templates, new verification techniques, better decomposition of Lane C tasks. This is Anna herself improving — the same pattern as the design source, where an assistant's *skills* generalize across the people she serves while each person's *memory* never does.

Craft compounds globally by design: it contains no client's content (§27.4's invariant), so it can live in the shared, controlled skill library (§13.1) and ship to every tenant as product improvement. Every customer benefits from Anna's total experience; no customer's data moved to make that true.

27.4 Governance — the promotion pipeline and the hard invariant

Nothing crosses from a tenant into Kind 2 or Kind 3 by drift. The path is a pipeline:

1. **Experience inside a tenant** — an outcome, a correction, a reviewer response, a technique that worked.
2. **Local abstraction** — the local layer (§24.2, Lane A) strips identity and fact, producing either a de-identified pattern candidate (Kind 2) or a generalized technique candidate (Kind 3).
3. **Human-reviewed promotion** — a named human reviews the candidate against the invariant and approves or rejects it. Promotion is a change-control event (§9): shared skills and templates are controlled documents (§13.1), so a promoted playbook arrives at tenants as a versioned update with a record of what changed and why.
4. **Receipt by tenants** — as skill/template versions and pattern-informed defaults, inside each tenant's own walls.

Hard invariant: no shared artifact — pattern, playbook, template, or default — may encode a client's facts, identity, or strategy. The promotion review exists to test exactly this, and a candidate that cannot be stated without a client's specifics is rejected or kept tenant-local under Kind 1. Per-customer controls sit in the policy engine (§24.3): contribute patterns / contribute nothing, set by the tenant admin, changeable, and audited (§17.7).

---

28. Why this wins (v0.5 addition)

Anna's craft compounds globally; your data never does. Every new customer starts with an Anna who has already learned from thousands of submissions' worth of pattern and craft — and their own data is walled off from day one, their rules are theirs (§21), their accounts are theirs (§17). That is the network effect regulated industries have never been offered, because every previous version of it was priced in custody. This architecture gets the compounding without the price: the thing that travels is how Anna works, and the thing that never travels is what the client owns.

---

29. Build implications — three kinds of learning (v0.5 addition)

Later-phase work (not in the §14.4 spike scope; the spike's job is the walls, §§19/23/26):

1. **De-identification gate service** in the local layer, shared with Lane C derivative production (§24.2) — one tokenization/abstraction implementation, two callers.
2. **Pattern-candidate store + cohort thresholding** (§27.2): candidates accumulate per pattern class and become promotable only at cohort minimum.
3. **Promotion workflow** as QMS change control (§9) with the §27.4 invariant as a review checklist on the ChangeControl object; promoted items land as versioned skill/template updates (§13.1).
4. **Contribution controls + reporting**: the TenantDataPolicy (§24.3) gains the contribute-patterns setting; the tenant audit view (§17.7) gains the pattern-class contribution report.

---

30. Agency-feedback intelligence — learning why agencies push back (v0.5 addition)

This part extends §27: of everything that can legitimately be learned across customers, **agency feedback is the highest-value source** — because it is the agencies' behavior, not any client's content, and because it is where approvals are actually won and lost. It also extends §10.3: the Correspondence intake already classifies agency letters and starts clocks; this part makes the platform *learn* from them, in two registers — the objective and the subjective.

30.1 The feedback corpus

Every agency communication received through the platform becomes structured data, not a filed PDF: FDA Refuse-to-Accept and Refuse-to-File notices, Complete Response Letters, Additional Information and interactive-review requests, deficiency lists, pre-submission (Q-Sub) written feedback, meeting minutes, and discipline review letters — and the EMA, MHRA, and Health Canada equivalents (§17.9).

```
FeedbackRecord { id, tenant_id, correspondence_ref,          // §10.3
object — the letter itself stays tenant content
                 agency, division_office, product_type,
```

```
application_type,
                  kind: "RTA" | "RTF" | "CRL" | "AI_REQUEST" |
"DEFICIENCY" | "QSUB_FEEDBACK"
                       | "MEETING_MINUTES" | "DISCIPLINE_LETTER" | ...,
                  items: [FeedbackItem] }
FeedbackItem   { id, feedback_ref, register: "objective" |
"subjective",
                  deficiency_class, detail, evidence_span,      // span
in the letter, per §10.3 discipline
                  confidence, coded_by, human_confirmed: bool }
```

Each record is tagged by agency, division/office, product type, and application type, so patterns can be read at exactly the level where review behavior differs.

30.2 Objective vs. subjective coding — the core distinction

- **Objective deficiencies** — missing data, failed specifications, unmet guidance requirements, format and validation failures — are coded as *checkable facts*. Each maps to a requirement, a checklist item, or an atom-level check the platform can run deterministically (§1, §10.1). An objective pattern, once coded, becomes a gate.
- **Subjective patterns** — coded separately, as their own register: reviewer- and division-level expectations. How a division likes risk framed. Document organization and navigation preferences. Data-presentation norms. Sensitivity to precedent arguments. Clarity and tone issues. The unwritten rules that experienced regulatory consultants carry in their heads and junior teams learn by getting letters. This tacit layer is what this part turns into a system: coded with its evidence span and a confidence, human-confirmed before it drives anything (§10.3's date discipline applies — a subjective coding is an inference about expectations, never quoted as the agency's stated policy).

30.3 The anticipation engine — clean-file review

Before any submission transmits, Anna runs the package against the learned feedback patterns for that agency, division/office, and product type:

1. **Objective checklist** — every coded objective pattern for the lane becomes a check (data present, specs met, guidance requirements answered, format valid). Failures are blocker Findings (§2.6).
2. **Subjective-fit review** — structure, framing, and presentation are reviewed against the division's coded expectations: is risk framed the way this division reads it, is the package navigable the way its reviewers navigate, are the precedent arguments the kind this division credits. Mismatches raise

Findings *while they can still be fixed* — before transmission, not in a letter six months later.

The clean-file review report joins the approval package (§17.6), so the approver sees the anticipated pushback alongside the validation report. The goal state, stated plainly: **fewer RTFs and deficiency rounds, faster approvals** — measured per customer as *deficiencies per submission, trending down over time*, with RTF/RTA rate and review rounds to approval as the companion metrics (§12.6 reporting).

30.4 The learning loop — three scopes, again

Each agency response updates three records, mirroring §21's scopes:

- **Project** — this application's history: what this division asked, what was answered, what it accepted (§21.1 project rules derive from it — a reviewer's stated expectation becomes a rule for every later sequence of this application).
- **Customer** — how this sponsor's products get received: recurring weaknesses in this sponsor's packages, strengths reviewers credit. Tenant-scoped, like everything of the customer's (§17.1).
- **Global pattern pool** — division tendencies, deficiency-class frequencies, subjective expectations: the §27.2 Kind-2 pool, fed by its richest source. Agency behavior is not client content; a division's expectations belong to no sponsor.

30.5 Source discipline

FDA correspondence received by a client belongs to the client — Kind 1 content (§27.1), walled in the tenant, full stop. What may enter the shared pool through the de-identification gate (§27.2) is only the *extracted pattern*: deficiency classes, division-level tendencies, presentation expectations — with no client facts, no product identity, and no reversible path back. One refinement: reviewer-level observations are coded at **division/office level** for the shared pool; anything attributable to an individual reviewer stays tenant-local, where it serves that customer's next filing and no one else's inferences.

---

31. Why this wins (v0.5 addition)

Every experienced regulatory consultant sells, in large part, a memory of agency behavior — what this division hates, how that office reads risk, which argument

landed last time. That memory is normally personal, perishable, and priced by the hour. This architecture makes it cumulative and systematic: every letter every customer receives through the platform sharpens the clean-file review for every customer (§30.3), while the letters themselves never leave their owners (§30.5). Customers file cleaner documents and get approved faster, measurably (deficiencies per submission, §30.3); the platform's advantage compounds with every review cycle in every tenant — the §28 network effect, running on the most decision-relevant data there is.

---

32. Build implications — agency-feedback intelligence (v0.5 addition)

Sequenced after the walls (§§19, 23, 26, 29), with the correspondence phase of §11 (phase 7):

1. **Corpus schema + coding workflow:** FeedbackRecord/FeedbackItem land with the §10.3 Correspondence intake; the classifier proposes kind, items, and register; a human confirms — subjective codings (§30.2) are never auto-promoted, because a wrong "unwritten rule" taught to the system is worse than none.
2. **Clean-file review as a `ready`-state gate:** the Submission state machine (§10.2) gains the §30.3 review as a required artifact before `ready`; objective checks first (deterministic, §10.1 pattern), subjective-fit review second, once enough confirmed patterns exist per division to make it honest — the review reports "insufficient pattern history" rather than inventing expectations.
3. **Metrics:** deficiencies-per-submission and RTF/RTA tracking per tenant (§12.6), with the global-pool contribution flowing through the §27.2 gate and the §27.4 promotion review.